CLAUDE CERTIFIED ARCHITECT — FOUNDATIONS

Section 1: Agentic Loops, Parallel Execution & Context Management

Exam objectives covered

1. Configure agentic loops to emit multiple tool calls within a single response turn, enabling parallel execution of independent subtasks and reducing total round-trip latency.
2. Apply session resumption techniques—including targeted re-analysis of changed files and context injection—to restore agent state accurately without repeating prior work.
3. Apply context management strategies—subagent isolation, scratchpad files, and targeted file reading—to sustain coherent codebase exploration across sessions exceeding context limits.
4. Apply context window optimization techniques—summarization, sliding windows, structured state objects, and selective retention—to maintain response quality as conversations exceed practical token limits.

Q1.

An architect wants an agent to fetch three unrelated API specs and compare them in the same turn. Which configuration best supports the exam objective for agentic loops?

A. Force the agent to call one tool, wait for the result, then decide on the next tool B. Allow the agent to emit multiple independent tool calls in a single response turn C. Disable tool use and require the agent to answer from prior memory D. Increase the maximum token output and keep tool calls sequential

Answer: B **Explanation:** Emitting multiple independent tool calls in one turn enables parallel execution and reduces total latency. A and D preserve sequential bottlenecks, while C prevents the agent from gathering needed external information.

Q2.

Which task is the **best** candidate for parallel tool execution?

A. Generate a migration plan after first reading a schema diff, then updating the plan based on test failures B. Open the same file repeatedly and rewrite it section by section in order C. Search three unrelated directories for deprecated API usage D. Run a build, then only if it fails, inspect the failing module

Answer: C **Explanation:** Searching unrelated directories is independent work, so it can be parallelized safely. A and D contain clear dependencies, and B operates on the same artifact in a way that is not safely independent.

Q3.

A team enables aggressive parallelism for code edits across many files. What is the **primary** risk they should evaluate first?

A. Increased context window size B. Conflicting writes to shared state or tightly coupled files C. The agent becoming unable to summarize results D. Reduced need for session resumption

Answer: B **Explanation:** Parallel execution is most valuable for independent subtasks, but it can introduce race conditions or inconsistent edits when shared state is involved. A, C, and D are not the main architectural risk of concurrent modification.

Q4.

An agent resumes a prior coding session after a crash. Which recovery approach best avoids wasted tokens and duplicate work?

A. Re-read the full repository and replay the entire conversation from the beginning B. Start over with a fresh plan and ignore prior artifacts C. Inject the prior summary/state and re-analyze only files changed since the last checkpoint D. Ask the user to restate every earlier decision in full

Answer: C **Explanation:** Session resumption should restore state efficiently by combining stored context with targeted re-analysis of changed files. A and D waste time and tokens, while B discards useful prior work.

Q5.

Which artifact is most useful to inject during session resumption?

A. A raw dump of every token from the previous conversation B. A structured state object with goals, completed steps, open questions, and relevant file references C. A list of all files in the repository regardless of relevance D. A random sample of prior assistant replies

Answer: B **Explanation:** A structured state object preserves the working set in a compact, reusable form. A is too expensive, C is noisy, and D is incomplete and unreliable for accurate restoration.

Q6.

A long-running investigation exceeds the model's practical context window. Which technique best supports coherent exploration across sessions?

A. Keep everything in live conversational memory and avoid external state B. Use subagent isolation for separate threads and persist concise scratchpad notes for handoff C. Re-open every previously viewed file on each turn D. Reduce the task scope until no tools are needed

Answer: B **Explanation:** Subagent isolation prevents unrelated threads from contaminating each other, and scratchpad notes preserve key findings across resumptions. A and C waste context, while D may avoid the problem rather than solve it.

Q7.

Why is targeted file reading preferred over repeatedly loading whole directories into context?

A. It guarantees the final answer is always shorter B. It minimizes irrelevant tokens and keeps attention on decision-critical evidence C. It removes the need for tool outputs entirely D. It ensures no summarization is ever required

Answer: B **Explanation:** Targeted reading improves context efficiency by focusing only on relevant code and evidence. A is not guaranteed, C is false, and D is unrealistic in long workflows.

Q8.

Which example best illustrates a **sliding window** context strategy?

A. Persisting only the most recent relevant turns plus durable summaries of earlier decisions B. Deleting all old context whenever the conversation gets long C. Copying the entire transcript into every new prompt D. Keeping only tool outputs and dropping all reasoning state

Answer: A **Explanation:** A sliding window retains recent details while compressing older but still important context into summaries. B loses critical history, C is inefficient, and D may omit user intent or decision rationale.

Q9.

An engineer stores a "state.json" file containing resolved assumptions, pending tasks, and touched files. What context-management role does this file primarily serve?

A. It replaces all need for model reasoning B. It acts as a structured scratchpad for session continuity and controlled context injection C. It is mainly useful for parallelizing unrelated tool calls D. It prevents tools from returning stale results

Answer: B **Explanation:** A structured scratchpad externalizes durable working memory and supports efficient resumption. It does not eliminate reasoning, directly control tool freshness, or inherently create parallelism.

Q10.

Which resumption plan is **most accurate** after the user says, "Only the auth module changed since yesterday"?

A. Re-index every package and rerun all prior searches across the repo B. Inject yesterday's summary, verify the auth-related files changed, and re-read only those plus their immediate dependencies C. Ignore the prior summary because it might bias the model D. Recreate the original prompt but omit tool outputs to save tokens

Answer: B **Explanation:** This approach combines preserved state with targeted validation and scoped re-analysis. A is wasteful, C throws away valuable context, and D risks missing key facts from prior tool results.

Q11.

Which option best demonstrates **selective retention**?

A. Keeping every intermediate chain-of-thought token forever B. Retaining final decisions, constraints, and unresolved issues while discarding repetitive or superseded detail C. Deleting all tool results once a summary exists D. Keeping only the most recent user message

Answer: B **Explanation:** Selective retention preserves the information most likely to matter later while pruning low-value repetition. A is inefficient, C is too aggressive, and D loses essential continuity.

Q12.

A parent agent splits a large migration into UI, backend, and infra investigations handled separately. What is the main context benefit of this design?

A. Each subagent can maintain a narrower working set, reducing interference between unrelated details B. It guarantees lower token usage than any other design C. It eliminates the need for final synthesis D. It ensures all subtasks can edit the same files safely at once

Answer: A **Explanation:** Subagent isolation keeps each thread focused and prevents unrelated facts from polluting local reasoning. B is not guaranteed, C is false, and D ignores coordination risks.

Q13.

Which scenario most clearly calls for summarization rather than raw transcript retention?

A. A short exchange with one file read and no follow-up B. A multi-hour debugging session with repeated hypotheses, dead ends, and several confirmed findings C. A single API lookup performed in one turn D. A trivial spelling correction in documentation

Answer: B **Explanation:** Long sessions accumulate noise, making summary compression essential for preserving only durable, actionable findings. The other scenarios are small enough that summarization is less critical.

Q14.

An agent has 15% context remaining and still needs to continue a codebase investigation. What is the best next step?

A. Keep appending full tool outputs until the window is exhausted B. Compact prior progress into a concise summary/state object and retain only active evidence C. Restart the session and lose prior work D. Stop using tools and answer from guesswork

Answer: B **Explanation:** When the context window is filling, the correct move is to compress prior state and keep only what remains operationally relevant. A degrades quality, C is wasteful, and D harms correctness.

Q15.

Which statement about parallel tool calls is most accurate?

A. Parallelism should be used whenever more than one tool exists B. Parallelism is ideal only when subtasks are independent and their outputs do not gate one another C. Parallelism is mainly a

substitute for context management D. Parallelism removes the need for session checkpoints

Answer: B **Explanation:** The key criterion is independence: if results do not depend on each other, parallelism reduces round trips safely. A is overbroad, and C and D confuse execution strategy with memory/state strategy.

Q16.

A resumed session produces inconsistent conclusions because the agent relied on an old summary after the code changed. What control would best prevent this?

A. Larger output limits B. Automatic re-analysis of the entire repo on every resume C. Change-aware validation that re-reads modified files before trusting stored summaries D. Disabling summaries entirely

Answer: C **Explanation:** Stored summaries are useful, but they must be validated against changed artifacts to avoid stale reasoning. A does not solve correctness, B is too expensive, and D throws away a valuable optimization.